

TEMA 7. Aspectos funcionales

1. ¿Qué es un puntero a una función? Pon un ejemplo de código en C, donde se define una función y que reciba una cadena de caracteres como parámetro y devuelva la cadena en mayúsculas. Crea un puntero en una variable local a dicha función llamado `aMayusculas` e invócala con el puntero.

Respuesta

Un **puntero a función** en C es una variable que almacena la dirección de memoria de una función. Permite invocar funciones de forma indirecta, pasando funciones como parámetros o almacenándolas en estructuras. Es un mecanismo básico para simular comportamiento flexible antes de la programación orientada a objetos.

En C, no existe el concepto de funciones como valores de primera clase, pero los punteros a función permiten aproximarse a esa idea. Esto resulta útil para diseñar código más genérico, por ejemplo, al aplicar distintas operaciones sobre datos sin cambiar la estructura principal del programa.

```
#include <stdio.h>
#include <ctype.h>

void aMayusculas(char* str) {
    for (int i = 0; str[i]; i++) {
        str[i] = toupper(str[i]);
    }
}

int main() {
    char texto[] = "hola";

    void (*func)(char*) = aMayusculas;
    func(texto);

    printf("%s\n", texto);
    return 0;
}
```

2. ¿Qué es una **función lambda** en un lenguaje de programación? Pon un ejemplo similar al anterior en Javascript y otro en Java con funciones lambda. Usa una variable local `aMayusculas` para apuntar a la función lambda. Por simplicidad, en Java, emplea `Function<String, String>` para el tipo de la referencia a la función lambda.

Respuesta

Una función lambda es una función anónima que puede definirse directamente en una expresión y asignarse a una variable o pasarse como parámetro. Permite escribir funciones de forma más compacta y flexible, especialmente en contextos donde se necesitan pequeñas transformaciones.

En lenguajes como JavaScript o Java, las funciones lambda son ciudadanos de primera clase o están muy integradas, lo que permite tratarlas como valores. Esto simplifica el código y evita tener que definir funciones completas con nombre.

```
let aMayusculas = (str) => str.toUpperCase();
console.log(aMayusculas("hola"));

import java.util.function.Function;

Function<String, String> aMayusculas = s -> s.toUpperCase();
System.out.println(aMayusculas.apply("hola"));
```

3. ¿Qué es el **paradigma funcional**? ¿Por qué a algunos lenguajes orientados a objetos como Java 8, se les llama multi-paradigma? ¿Qué quiere decir que las funciones son **ciudadanos de primera clase**?

Respuesta

El paradigma funcional es un estilo de programación en el que las funciones son el elemento principal. Se basa en evitar estados mutables y efectos secundarios, y en usar funciones puras que siempre producen el mismo resultado para los mismos parámetros.

Lenguajes como Java se consideran multi-paradigma porque permiten combinar programación orientada a objetos con programación funcional. Desde Java 8, se incorporaron lambdas y otras características funcionales.

Decir que las funciones son “ciudadanos de primera clase” significa que pueden asignarse a variables, pasarse como parámetros y devolverse como resultado, igual que cualquier otro dato.

4. Explica la sintaxis básica de una función lambda en Java.

Respuesta

La sintaxis básica de una lambda en Java es: (parámetros) -> expresión o (parámetros) -> { bloque }. Se omiten tipos cuando el compilador puede inferirlos.

Las lambdas se usan normalmente junto con interfaces funcionales. El compilador utiliza el tipo de la interfaz para determinar el tipo de la lambda.

Por ejemplo, (s) -> s.toUpperCase() es una lambda que recibe un String y devuelve otro String.

5. Ahora recibamos una función como parámetro a un método y la llamaremos desde dentro. Amplia los ejemplos anteriores de Java y JavaScript con un método llamado `transformar`, que reciba un `String` como parámetro y luego una función transformadora como lo es `aMayúsculas` y la invoque desde dentro.

Respuesta

Se puede pasar una función como parámetro a un método, lo que permite aplicar comportamientos distintos sin modificar el método.

```
import java.util.function.Function;

public static String transformar(String s, Function<String, String> f) {
    return f.apply(s);
}

function transformar(s, f) {
    return f(s);
}
```

Esto permite reutilizar el método con distintas funciones transformadoras.

6. Ahora, invoca `transformar`, con una nueva función lambda directamente en la llamada a `transformar`, por ejemplo, una función lambda que invierta la cadena. Define la función de inversión justo cuando la estás pasando como parámetro.

Respuesta

Se puede pasar directamente una lambda como parámetro **sin** necesidad de almacenarla en una variable. Esto es muy útil cuando la función solo se usa una vez.

```
transformar("hola", s -> new StringBuilder(s).reverse().toString());
transformar("hola", s => s.split("").reverse().join(""));
```

Esto hace el código más compacto y expresivo.

7. ¿Qué se entiende por cierre o "closure" en el contexto de las funciones lambda? Pon un ejemplo en Java de cómo una función lambda es capaz de acceder a una variable local en el contexto donde fue definida. Modifica el ejemplo anterior, creando otra función lambda para transformar una cadena, pero que lo que haga es concatenar a la cadena de entrada otra cadena que está en una variable local definida fuera de la función lambda.

Respuesta

Un closure es una función que captura variables del entorno donde fue creada. Esto significa que la lambda puede usar variables externas incluso después de que ese contexto haya terminado.

En Java, las variables capturadas deben ser efectivamente finales.

```
String sufijo = "!!!";

Function<String, String> f = s -> s + sufijo;
System.out.println(f.apply("hola"));
```

Aquí la lambda accede a sufijo, aunque no sea un parámetro.

8. Reflexiona: ¿en qué se diferencia entonces una función lambda de los punteros a funciones que hay en C?

Respuesta

Los punteros a función en C solo almacenan direcciones de funciones. No capturan

variables del entorno ni permiten closures.

Las lambdas son más potentes porque pueden capturar contexto, ser anónimas y definirse `inline`. Además, están integradas en el sistema de tipos del lenguaje.

Por tanto, las lambdas permiten un estilo de programación más flexible y expresivo.

9. Devolvamos ahora funciones. Creemos ahora una función que sea capaz de crear funciones "descuento". Una función "descuento", decrementa un porcentaje pasado como parámetro. Por simplicidad, usa `Function<Double, Double>` para su tipo. La función `crearDescuento(porcentaje)`, recibe solo el porcentaje de descuento a aplicar y devuelve la función de descuento. Prueba a crear dos descuentos distintos y aplicarlos a una cantidad. Explica la closure en la función descuento.

Respuesta

Se pueden devolver funciones desde otras funciones, lo que permite construir funciones personalizadas.

```
import java.util.function.Function;

public static Function<Double, Double> crearDescuento(double porcentaje) {
    return precio -> precio * (1 - porcentaje);
}

Function<Double, Double> d1 = crearDescuento(0.1);
System.out.println(d1.apply(100.0));
```

La lambda captura porcentaje, lo que es un ejemplo de closure.

10. En Java, que es un lenguaje con comprobación estática de tipos, donde los tipos se declaran, toda función lambda tiene un tipo, que se conoce como `**interfaz funcional**`. ¿Qué es una `**interfaz funcional**`? ¿Qué requisitos tiene?

Respuesta

Una interfaz funcional es una interfaz con un único método abstracto. Es el tipo que permite definir lambdas en Java.

Puede tener otros métodos por defecto o estáticos, pero solo uno abstracto.

Se usa como tipo objetivo para expresiones lambda.

11. Creemos una interfaz funcional a mano. Por ejemplo, define la interfaz funcional del ejemplo que transforma la cadena en otra. Llámale `Transformador`, que define una función que convierte una cadena de `texto` (`String`) en `otra` (`String`).

Respuesta

```
@FunctionalInterface
interface Transformador {
    String transformar(String s);
}
```

Esta interfaz permite definir lambdas que transformen cadenas.

12. Ahora hagamos la interfaz funcional algo más genérica y empleando generics, para que permita definir un `Transformador` de un tipo en otro. Pon un ejemplo de un transformador que redondea un `Double` en un `Integer`.

Respuesta

```
@FunctionalInterface
interface Transformador<T, R> {
    R transformar(T t);
}
Transformador<Double, Integer> redondear = d -> (int) Math.round(d);
```

Esto permite reutilizar la interfaz para distintos tipos.

13. `Transformador`, en su versión genérica, parece muy útil y reutilizable, hasta el punto de que es igual a una interfaz funcional que ya hay, que es `Function<T, R>`. Muestra las interfaces funcionales predefinidas que hay en Java.

Respuesta

Java proporciona interfaces como:

```
Function<T, R>
Consumer<T>
Supplier<T>
Predicate<T>
```

Estas cubren la mayoría de casos comunes y evitan definir nuevas interfaces.

14. Vamos a ver ejemplos expresivos de funcional en Java. Estudiemos el `List.forEach`, como versión funcional del bucle `for`. Emplea el `forEach` para recorrer una lista de `Integer` y que muestre un mensaje si el entero es positivo.

Respuesta

```
import java.util.*;

List<Integer> lista = Arrays.asList(1, -2, 3);

lista.forEach(n -> {
    if (n > 0) {
        System.out.println("Positivo: " + n);
    }
});
```

Esto sustituye el bucle `for` por una forma funcional.

15. Repasando el tema de genericidad, fíjate en la firma de `forEach`, ¿por qué se usa `Consumer<? super T>` y no `Consumer<T>`? Explica qué significa **PECS**, y explícalo para el caso de mejorar el ejemplo del método `transformar` la hora de definir el tipo de la función transformadora.

Respuesta

PECS significa: Producer Extends, Consumer Super.

Se usa ? extends T cuando se leen datos y ? super T cuando se escriben.

En forEach, se usa Consumer<? super T> porque consume elementos.

16. Referencias a métodos. Podemos obtener una referencia a métodos de objetos o clases. Pon un ejemplo en JavaScript y en Java, de una clase `Persona` con un método `saludar`. En el código principal, crea una `Persona` con un nombre, y obtén una referencia a su método `saludar` en una variable local. Invoca `saludar` con esa referencia a su método `saludar`.

Respuesta

```
class Persona {
    String nombre;

    Persona(String n) { nombre = n; }

    void saludar() {
        System.out.println("Hola " + nombre);
    }
}

Persona p = new Persona("Ana");
Runnable r = p::saludar;
r.run();

class Persona {
    constructor(nombre) { this.nombre = nombre; }
    saludar() { console.log("Hola " + this.nombre); }
}

let p = new Persona("Ana");
let f = p.saludar.bind(p);
f();
```

17. ¿Qué tipos de referencias a método se pueden hacer en Java? Pon un ejemplo de referencia a método estático, a constructor, a método de instancia de una instancia concreta y a método de instancia sobre cualquier instancia.

Respuesta

```
// Estático
Function<String, Integer> f1 = Integer::parseInt;

// Constructor
Supplier<List<String>> f2 = ArrayList::new;

// Instancia concreta
Persona p = new Persona("Ana");
Runnable f3 = p::saludar;

// Instancia general
Function<String, String> f4 = String::toUpperCase;
```

18. Otro ejemplo expresivo. Ordena una lista de `Persona`, cada persona tiene un nombre y una edad (de tipo entero). Ordena la lista de `Persona` con `Collections.sort`, pasándole como comparador una expresión lambda que compare la edad de ambas personas y si tienen la misma edad, se ordene por orden alfabético del nombre. Crea dos versiones: Una con la función de comparación hecha manualmente, y otra empleando `Comparator`.

Respuesta

```
Collections.sort(lista, (p1, p2) -> {
    int cmp = Integer.compare(p1.edad, p2.edad);
    if (cmp == 0) return p1.nombre.compareTo(p2.nombre);
    return cmp;
});
Collections.sort(lista,
    Comparator.comparingInt((Persona p) -> p.edad)
        .thenComparing(p -> p.nombre));
```